# MIT 8103 – Advanced Database Systems

## Final Individual Practical Portfolio Summary

**Student Name:** Emmanuel Eke

**Matriculation Number:** 2026/A/MIT/0557

**Case Study:** FlexPay Financial Technologies

**Academic Session:** 2026/2027 First Semester

**GitHub Repository:** https://github.com/emmanuelkaluke/Advanced-Database-Portfolio

### 1. Introduction

This portfolio covers five practical activities completed as part of the Advanced Database Systems course. The work was based on a fictional Nigerian fintech company called FlexPay Financial Technologies, which provides digital financial services such as customer account management, deposits, withdrawals, transfers and transaction processing.

I chose to use one case study throughout the activities so that I could see how the different database concepts connect in a real-world business scenario.

The practical work involved PostgreSQL, MongoDB Atlas, JavaScript and other development tools. Each activity focused on a different aspect of database systems, from designing a database to understanding how distributed databases operate.

### 2. Summary of Activities and Implementation

#### Activity 1: Database Design and Modelling

The first activity involved designing the database structure for FlexPay. I identified three main entities: customers, accounts and transactions.

I created an Entity Relationship Diagram and converted the design into a relational schema. PostgreSQL was used to implement the tables, relationships and constraints.

Primary keys, foreign keys, unique constraints and check constraints were applied to help maintain data accuracy. I also considered normalisation and organised the database up to Third Normal Form (3NF).

This exercise helped me understand the importance of planning a database structure before implementing it, particularly when dealing with financial information.

#### Activity 2: Query Processing and Optimisation

The second activity focused on retrieving information and improving query performance.

I wrote SQL queries involving filtering, sorting, joins, aggregate functions and subqueries. I also used PostgreSQL's `EXPLAIN ANALYZE` feature to examine how the database executed queries.

One important part of this exercise was comparing query performance before and after creating a composite index on `account_id` and `transaction_date`. I generated 10,000 additional transaction records to test the query performance.

The execution time decreased from 2.593 milliseconds to 1.354 milliseconds, representing approximately a 47.8% reduction in the measured execution time.

This exercise showed me how indexing can improve data retrieval, especially when working with tables containing a large number of records.

#### Activity 3: Transactions and Concurrency

The third activity explored how databases manage operations that must be completed reliably.

Using FlexPay account operations, I tested deposits, withdrawals and internal transfers. I also practised using `COMMIT` and `ROLLBACK` to control database transactions.

A rollback exercise demonstrated that an uncommitted change could be reversed without permanently affecting the account balance.

I also tested concurrency using two PostgreSQL sessions. One session locked an account row using `SELECT FOR UPDATE`, while the second session attempted to update the same account. The second operation waited until the first transaction committed.

This helped me understand how row-level locking prevents conflicting updates and why transaction management is important in financial systems.

### Activity 4: NoSQL and Advanced Data Models

For the fourth activity, I used MongoDB Atlas to explore document-oriented database management.

I created a MongoDB database named `flexpay_nosql` and worked with a collection containing sample transaction documents.

The practical work included inserting and updating documents, filtering records, performing aggregation operations and creating a unique index.

Unlike PostgreSQL, which stores information in related tables, MongoDB uses a document-based structure. Working with both technologies helped me understand their different approaches and where each can be useful.

### Activity 5: Distributed and Cloud Database Exercise

The final activity focused on distributed database concepts using MongoDB Atlas.

I inspected the cluster's replica set configuration and confirmed that it contained three members. I also carried out practical tests involving majority write concern and majority read concern.

A separate test successfully retrieved a transaction using the `secondaryPreferred` read preference. Although the operation succeeded, the output did not establish whether the read was actually served by a secondary member.

I also developed a JavaScript simulation to demonstrate horizontal partitioning. Nine sample transactions were distributed across three logical shards, with three transactions assigned to each shard.

The sharding exercise was a simulation rather than an actual MongoDB sharded deployment. The Atlas cluster used for the practical work was a replica set.

## 3. Overall Learning and Reflection

Working through the five activities gave me a better understanding of how database systems are designed, implemented and managed.

One of the most useful parts of the project was working with PostgreSQL and MongoDB side by side. Before carrying out the exercises, I understood databases mainly in terms of storing and retrieving information. The practical work helped me appreciate the importance of database structure, data integrity, performance and reliability.

The query optimisation exercise was particularly useful because I could compare the actual execution plans and see the effect of adding an index.

The transaction and concurrency exercise also helped me understand why financial applications need proper transaction controls. A small mistake in handling account updates could result in incorrect balances or inconsistent records.

Working with MongoDB Atlas introduced me to cloud database services and distributed database concepts. It also helped me understand that replication, read preferences and sharding are different concepts, each serving a particular purpose.

I also gained more experience using the Terminal, VS Code, Node.js, Git and GitHub to organise and manage a technical project.

Overall, the project improved my practical understanding of database systems and showed me how the concepts covered in class can be applied to a financial technology platform.

## 4. Conclusion

The FlexPay portfolio brought together the major areas covered in the Advanced Database Systems course.

Through the five activities, I designed a relational database, wrote and optimised SQL queries, tested transaction management and concurrency, implemented NoSQL operations and explored distributed database concepts using MongoDB Atlas.

The exercises demonstrated that choosing and managing a database involves more than simply storing information. Data integrity, performance, availability, consistency and scalability must also be considered.

The experience has given me a stronger foundation for understanding how databases support modern applications, particularly systems that handle financial transactions.

## 5. References


1. PostgreSQL Global Development Group. *PostgreSQL Documentation*. https://www.postgresql.org/docs/
2. MongoDB. *MongoDB Manual*. https://www.mongodb.com/docs/manual/
3. MongoDB. *MongoDB Node.js Driver Documentation*. https://www.mongodb.com/docs/drivers/node/current/
4. Git. *Git Documentation*. https://git-scm.com/doc
5. Mermaid. *Entity Relationship Diagrams Documentation*. https://mermaid.js.org/syntax/entityRelationshipDiagram.html


## 6. Declaration of AI Assistance

AI tools were used to support the understanding of database concepts, troubleshoot implementation issues, review code and assist with documentation.

The SQL and JavaScript exercises were executed in the development environment, and the practical outputs were reviewed against the results obtained. AI-generated explanations and documentation were also reviewed and adjusted to reflect the actual work completed.

I take responsibility for understanding and verifying the submitted implementation and documentation.